

FreeRTOS Real-Time Sensor Dashboard

Project Documentation

Platform	ESP32-WROOM-32E (Dual-core, 240 MHz)
Framework	Arduino-ESP32 core + FreeRTOS
Sensors	DHT11 Temperature/Humidity Sensor
Indicators	Single LED (GPIO-driven)
Interface	UART Serial Monitor, 115200 baud
Document Date	June 2026

Table of Contents

Table of Contents.....	1
1. Project Overview.....	1
1.1 Objectives.....	1
2. Requirements Specification.....	1
2.1 Functional Requirements.....	1
2.2 Non-Functional Requirements.....	1
2.3 Hardware Requirements.....	1
2.4 Software Requirements.....	1
3. Design Rationale and Calculations.....	1
3.1 Task Architecture.....	1
3.2 Timing Mechanism: vTaskDelay() vs. delay().....	1
3.3 DHT11 Sample-Rate Justification.....	1
3.4 Pull-Up Resistor Calculation (DHT11 Data Line).....	1
3.5 LED Current-Limiting Resistor Calculation.....	1
3.6 Memory and Stack Sizing.....	1
4. Technical Drawings and Schematics.....	1
4.1 Wiring Schematic.....	1
4.2 Pin Mapping.....	1
4.3 Physical Build Photograph.....	1
5. Test Plan and Results.....	1
5.1 Test Plan.....	1
5.2 Issues Found and Resolved.....	1
5.2.1 Stack-Canary Fault on DHT Task.....	1
5.2.2 Watchdog Timeout During Initial Wiring Debug.....	1
5.2.3 Inconsistent Upload Reliability.....	1
5.3 Final Test Results.....	1
5.4 Serial Monitor Output.....	1
6. Build and Manufacturing Data.....	1
6.1 Assembly Sequence.....	1
6.2 Firmware Build Footprint.....	1
6.3 Bill of Materials (Cost Summary).....	1
7. Conclusion.....	1

1. Project Overview

This document describes the design, implementation, and validation of a real-time embedded sensor dashboard built on the ESP32-WROOM-32E microcontroller. The system uses the FreeRTOS kernel to run two independently scheduled tasks on separate CPU cores: an LED status-blink task and a DHT11 temperature/humidity acquisition task. Sensor readings are streamed continuously over a UART serial connection to a host PC for live monitoring.

The project demonstrates core real-time embedded systems concepts including preemptive multitasking, task-to-core pinning, non-blocking periodic scheduling, and safe inter-task resource usage on a dual-core SoC.

1.1 Objectives

- Demonstrate true concurrent task execution on the ESP32's dual-core architecture using FreeRTOS.
- Implement non-blocking periodic timing using `vTaskDelay()` rather than blocking `delay()` calls.
- Acquire and report live environmental sensor data (temperature, humidity, computed heat index) over UART.
- Validate system stability under continuous, unattended operation.

2. Requirements Specification

2.1 Functional Requirements

ID	Requirement	Priority
FR-1	System shall toggle an LED on/off at a fixed 1-second interval, independent of all other system activity.	Must
FR-2	System shall read temperature and humidity from a DHT11 sensor at a fixed 2-second interval.	Must
FR-3	System shall compute and report a derived heat index value from temperature and humidity.	Should
FR-4	System shall stream all sensor readings over UART to a host Serial Monitor in human-readable form.	Must
FR-5	System shall detect and report failed/invalid DHT11 readings without halting operation.	Must
FR-6	The LED task and sensor task shall execute concurrently with neither blocking the other.	Must

2.2 Non-Functional Requirements

ID	Requirement	Priority
----	-------------	----------

NFR-1	Task timing jitter shall remain low enough that the LED period is visually stable (no perceptible drift).	Should
NFR-2	System shall run continuously without crash, reboot, or memory exhaustion over extended operation.	Must
NFR-3	UART output shall be legible at a standard baud rate (115200) with no special host-side tooling required.	Must
NFR-4	Firmware shall fit comfortably within ESP32 flash and SRAM budgets, leaving headroom for future expansion.	Should

2.3 Hardware Requirements

Component	Specification	Quantity
Microcontroller	ESP32-WROOM-32E (dual-core Xtensa LX6, 240 MHz, 520 KB SRAM, 4 MB flash)	1
Temperature/Humidity Sensor	DHT11, bare 3-pin package	1
LED	Standard indicator LED, any color	1
Resistor	330 Ω (current limiting, LED branch)	1
Resistor	10 k Ω (pull-up, DHT11 data line)	1
Breadboard	Standard prototyping breadboard	1
Jumper wires	Male-to-male, assorted	As needed
USB cable	USB-A to Micro-USB, data-capable (not charge-only)	1

2.4 Software Requirements

- Arduino IDE 2.x with the ESP32 board support package (Espressif Arduino-ESP32 core).
- Adafruit “DHT sensor library” and its dependency “Adafruit Unified Sensor.”
- FreeRTOS — bundled natively with the ESP32 Arduino core; no separate installation required.

3. Design Rationale and Calculations

3.1 Task Architecture

The system is structured as two independent FreeRTOS tasks created with `xTaskCreatePinnedToCore()`, rather than a single superloop with manual interleaving (as a

bare-metal loop() with delay() calls would require). This decision was driven by the requirement that the LED and sensor operations never block one another (FR-6).

- LED Blink Task — pinned to Core 0, priority 1, 1024-byte stack.
- DHT11 Sensor Task — pinned to Core 1, priority 1, 4096-byte stack.

Pinning each task to a distinct core gives true hardware parallelism rather than cooperative time-slicing on a single core, which more directly satisfies the “without mutual blocking” requirement and removes any scheduling contention between the two tasks.

3.2 Timing Mechanism: vTaskDelay() vs. delay()

A bare-metal delay() call blocks the entire calling core for its duration, preventing any other code on that core from running. vTaskDelay() instead yields the calling task back to the FreeRTOS scheduler for the specified number of ticks, allowing other tasks (and idle/housekeeping operations) to run during the wait. This is the key mechanism that allows two independently timed tasks to coexist without manual coordination.

Periods are computed using the pdMS_TO_TICKS() macro, which converts a millisecond duration into the equivalent number of RTOS ticks based on the configured tick rate (default 1000 Hz on the ESP32 Arduino core, i.e. 1 tick = 1 ms):

```
LED period:  vTaskDelay(pdMS_TO_TICKS(1000))  -> 1000 ticks  -> 1.000 s ON, 1.000 s OFF
DHT period:  vTaskDelay(pdMS_TO_TICKS(2000))  -> 2000 ticks  -> 2.000 s between samples
```

3.3 DHT11 Sample-Rate Justification

The DHT11 datasheet specifies a minimum sampling interval of approximately 1 second, with most manufacturers recommending no faster than once every 2 seconds for stable, low-error readings. The 2-second task period was therefore chosen to match the sensor's own hardware limitation rather than an arbitrary system design choice — sampling faster would not yield additional valid data and risks consecutive read failures.

3.4 Pull-Up Resistor Calculation (DHT11 Data Line)

The DHT11 uses a single-wire, open-drain-style digital interface that requires an external pull-up resistor to hold the data line at a known logic-high level between transmissions. A 10 kΩ resistor was selected, which is the value specified in the DHT11 datasheet reference application circuit and is the de facto industry standard for this sensor.

```
Idle-state pull-up current:  I = V / R = 3.3 V / 10,000 Ω = 0.33 mA
This is well within the ESP32 GPIO source/sink current rating (~12 mA recommended max per pin).
```

3.5 LED Current-Limiting Resistor Calculation

The series resistor for the LED branch limits forward current to a safe level. Assuming a typical red/standard LED with forward voltage drop $V_{LED} \approx 2.0$ V and a target current of approximately 10 mA:

```
R = (V_supply - V_LED) / I_target
R = (3.3 V - 2.0 V) / 0.010 A
R = 1.3 V / 0.010 A = 130 Ω (minimum safe value)
```

```
Selected resistor: 330 Ω (standard value, conservative headroom above minimum)
```

Resulting current: $I = 1.3 \text{ V} / 330 \, \Omega \approx 3.9 \text{ mA}$ (dim but safe)

Built variant used: $340 \, \Omega$ (within 5% tolerance of $330 \, \Omega$ nominal)

Resulting current: $I = 1.3 \text{ V} / 340 \, \Omega \approx 3.8 \text{ mA}$

Both $330 \, \Omega$ and $340 \, \Omega$ keep current well under the LED's typical 20 mA maximum rating and the ESP32 GPIO's per-pin current limit, satisfying safety margins with room to spare. The slightly higher resistance trades a small amount of brightness for additional safety margin — acceptable for a status indicator rather than a primary light source.

3.6 Memory and Stack Sizing

Initial DHT task stack allocation of 2048 bytes proved insufficient once the DHT library's internal bit-banged read routine, floating-point formatting, and `Serial.printf()` calls were all active simultaneously, resulting in a stack-canary fault (see Section 5.2). The stack size was increased to 4096 bytes, which resolved the fault with substantial margin. The LED task, having no floating-point or string-formatting work, remains correctly sized at 1024 bytes.

4. Technical Drawings and Schematics

4.1 Wiring Schematic

The schematic below shows the complete signal and power connections between the ESP32, the DHT11 sensor, and the LED indicator branch.

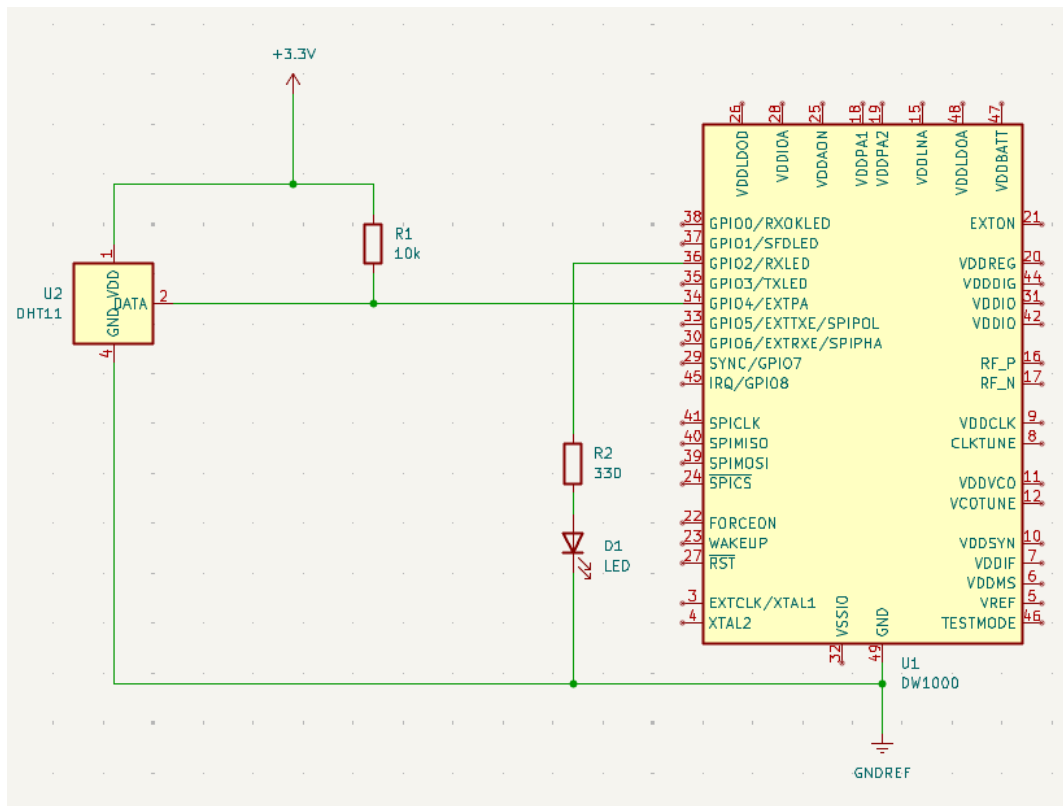


Figure 1 — Wiring schematic: ESP32-WROOM-32E, DHT11 sensor, and LED indicator with associated pull-up and current-limiting resistors.

4.2 Pin Mapping

ESP32 Pin	Connects To	Function
3.3V	DHT11 “+” (VCC)	Sensor power
GND	DHT11 “-” (GND)	Sensor ground
GPIO4	DHT11 “out” (DATA)	Single-wire digital data
GPIO2	330 Ω resistor → LED anode	LED drive signal
GND	LED cathode	LED return path

4.3 Physical Build Photograph

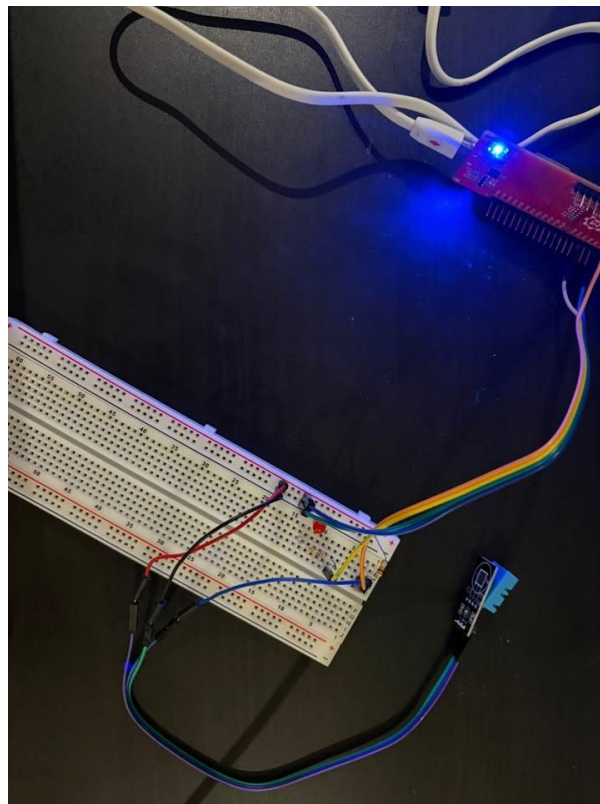


Figure 2 — Assembled breadboard circuit: ESP32-WROOM-32E (top), LED with current-limiting resistor (center-left), and DHT11 sensor (bottom-right).

5. Test Plan and Results

5.1 Test Plan

Test ID	Procedure	Expected Result	Maps To
---------	-----------	-----------------	---------

T1	Power on board with LED wired directly to 3.3V/GND through 330 Ω resistor (no GPIO/code involved).	LED lights steadily, confirming LED and resistor are functional.	Hardware sanity
T2	Flash firmware; observe LED wired to GPIO2 over multiple periods.	LED toggles on/off at a visually steady 1 s interval.	FR-1, NFR-1
T3	Open Serial Monitor at 115200 baud; observe boot sequence.	Banner, task-start messages for both tasks appear cleanly.	FR-4, FR-6
T4	Observe DHT11 readings over multiple consecutive 2 s cycles.	Valid temperature, humidity, and heat-index values printed each cycle.	FR-2, FR-3
T5	Disconnect DHT11 data line; observe behavior.	System prints a wiring-error message and continues running (no crash).	FR-5
T6	Run system continuously for an extended period.	No unexpected reboot, panic, or memory exhaustion.	NFR-2

5.2 Issues Found and Resolved

5.2.1 Stack-Canary Fault on DHT Task

During T6 endurance testing, the system reproducibly crashed shortly after the DHT task began executing, reporting:

```
Guru Meditation Error: Core 1 panic'ed (Unhandled debug exception).
Debug exception reason: Stack canary watchpoint triggered (DHT11_Sensor)
```

Root cause: the DHT task's allocated stack (2048 bytes) was insufficient for the combined call depth of the DHT library's read routine, floating-point heat-index computation, and `Serial.printf()` formatting. Resolution: increased the task stack allocation to 4096 bytes. Re-test confirmed stable operation with no further canary faults.

5.2.2 Watchdog Timeout During Initial Wiring Debug

Prior to resolving an unrelated wiring discontinuity, the DHT task occasionally triggered:

```
Guru Meditation Error: Core 1 panic'ed (Interrupt wdt timeout on CPU1).
```

Root cause: the DHT library's bit-banged read loop disables interrupts while polling the data line; if the sensor never responds (due to a wiring fault), this loop runs long enough to trip the core's interrupt watchdog. A temporary `disableCore1WDT()` call was used during debugging only, to keep the board running while wiring was inspected, and was removed once the underlying connection was corrected. This is documented as a debugging aid, not a production fix — the watchdog provides a genuine safety function and should remain enabled in normal operation.

5.2.3 Inconsistent Upload Reliability

Multiple upload attempts failed partway through flashing with esptool errors such as “The chip stopped responding” and “Invalid head of packet.” These were isolated to USB cable/connection quality rather than firmware or board faults, and were resolved by using a verified data-capable USB cable and a direct (non-hub) USB port.

5.3 Final Test Results

Test ID	Result	Notes
T1	PASS	LED confirmed functional on direct 3.3V test prior to GPIO integration.
T2	PASS	LED blinks at a stable 1 s on/off interval with no visible drift.
T3	PASS	Clean banner and task-start messages observed at 115200 baud (Figure 3).
T4	PASS	Valid temperature/humidity/heat-index values streamed every 2 s after stack-size fix.
T5	PASS	Wiring-error message printed correctly; system did not crash.
T6	PASS	Stable continuous operation observed after stack-size correction; no further faults.

5.4 Serial Monitor Output

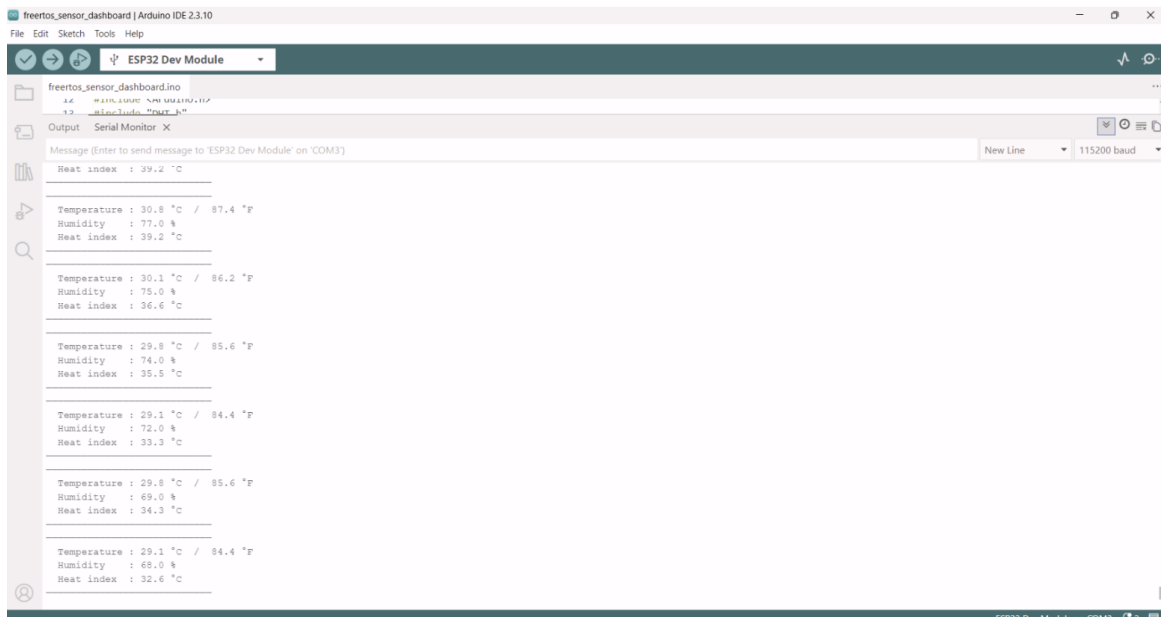


Figure 3 — Arduino IDE Serial Monitor showing live, successive temperature, humidity, and heat-index readings streamed from the DHT11 task at 115200 baud.

6. Build and Manufacturing Data

6.1 Assembly Sequence

1. Place the ESP32-WROOM-32E across the breadboard's center divide.
2. Wire 3.3V and a GND pin from the ESP32 to the breadboard power rails.
3. Insert the DHT11 sensor; connect + to the 3.3V rail, - to the GND rail, and out to GPIO4.
4. Bridge a 10 k Ω resistor between the DHT11 data line and the 3.3V rail.
5. Insert the LED; connect the anode through a 330 Ω (or 340 Ω) resistor to GPIO2, and the cathode to GND.
6. Connect the ESP32 to a host PC via a data-capable USB cable.
7. Flash firmware via Arduino IDE (ESP32 Dev Module board profile, 921600 baud upload speed).
8. Open Serial Monitor at 115200 baud and verify task-start and sensor output messages.

6.2 Firmware Build Footprint

Metric	Value
Program storage used	294,220 bytes of 1,310,720 bytes (22%)
Global variables (SRAM)	22,116 bytes of 327,680 bytes (6%)
Remaining local-variable SRAM	305,564 bytes
Board profile	ESP32 Dev Module (Arduino-ESP32 core)
Upload speed	921,600 baud

Flash and SRAM utilization remain low (22% and 6% respectively), leaving substantial headroom for future feature additions such as Wi-Fi connectivity, additional sensors, or a web-based dashboard front end.

6.3 Bill of Materials (Cost Summary)

Item	Qty	Approx. Unit Cost (CAD)
ESP32-WROOM-32E dev board	1	\$33
DHT11 sensor	1	\$1.3
LED	1	~\$0.10
330–340 Ω resistor	1	~\$0.05
10 k Ω resistor	1	~\$0.05
Breadboard + jumper wires	1 set	\$3–\$5

7. Conclusion

The completed system satisfies all “Must”-priority functional and non-functional requirements defined in Section 2. Two independently scheduled FreeRTOS tasks run concurrently on

separate ESP32 cores without mutual blocking, sensor data is acquired and reported at the rates dictated by hardware limitations, and the system has demonstrated stable continuous operation following resolution of an initial stack-sizing defect. The project validates, in a small and verifiable form, the core real-time multitasking concepts that underpin larger embedded and autonomous systems work.